6-5 · Kafka Connect와 MinIO 적재 · 이론 1H

Connect Sink 구조와 flush.size

코드 없는 컨슈머 — 언제 파일을 쓰고 어디에 두는가

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1시간 뒤에는

① 소스 커넥터와 싱크 커넥터의 방향 차이를 말할 수 있다

② MinIO 가 S3 API 호환이라 store.url 만 바꾸면 되는 이유를 안다

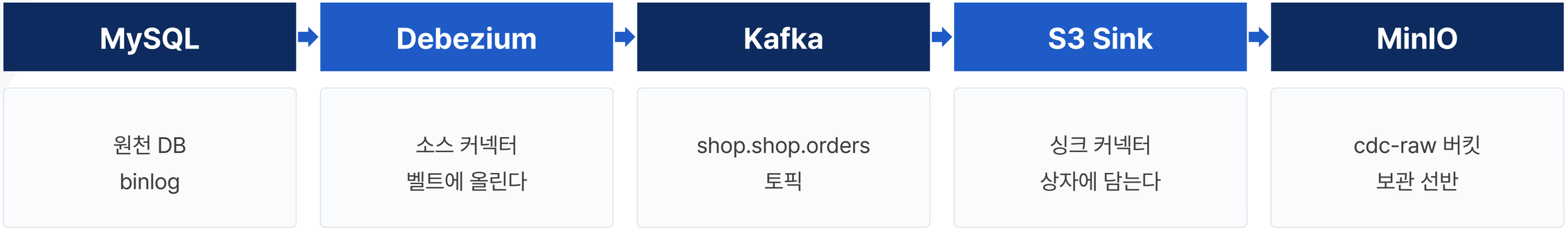
③ flush.size 만큼 모여야 파일이 생기는 것을 설명한다

④ 적재 경로 규칙과 raw 층의 뜻을 안다

다음 2.5H 실습에서 INSERT 5건을 넣어 파일이 생기는 순간을 눈으로 봅니다

1. 소스와 싱크

오늘 완성할 흐름 — 꺼내는 쪽과 내려놓는 쪽



6-4 는 DB 에서 Kafka 로 「꺼내는」 소스, 오늘은 Kafka 에서 스토리지로 「내려놓는」 싱크

컨베이어 벨트 — 소스는 창고에서 벨트에 올리는 사람, 싱크는 벨트 끝에서 상자에 담아 선반에 두는 사람

소스 커넥터와 싱크 커넥터 — 네 가지 차이

	소스 커넥터 (6-4)	싱크 커넥터 (오늘)
방향	밖 → Kafka	Kafka → 밖
예	Debezium MySQL	S3 Sink · JDBC Sink 등
설정에서	database.* 로 원천 지정	topics 로 구독 토픽 지정
오프셋	원천의 위치(binlog)	컨슈머 그룹 오프셋

싱크 커넥터는 사실 컨슈머 — 6-3 에서 직접 만든 Consumer 와 하는 일이 같고, 코드 대신 JSON 설정으로 부린다

2. MinIO

MinIO 는 S3 와 같은 말을 쓴다 — store.url 만 바꾼다

1. **S3 API 가 표준**
   4-1 에서 배운 것 — S3 와 같은 말(API)을 쓰는 스토리지가 많다

2. **MinIO 는 S3 호환 오브젝트 스토리지**
   로컬 Docker 로 S3 실습 환경을 대신한다

3. **store.url 이 갈림길**
   비워 두면 AWS S3, MinIO 주소를 넣으면 MinIO 로 간다

설정 한 줄

store.url

= MinIO 주소

3. flush.size

flush.size = 5 — 다섯 건이 모여야 파일 하나

1건 도착	2건 도착	3·4건	5건 도착
버퍼에 쌓음 파일 없음	버퍼에 쌓음 파일 없음	여전히 쌓기만	파일 하나로 쓴다

5건이 모이기 전까지는 아무 일도 안 일어난 것처럼 보인다 — 운영에서는 rotate.interval.ms(시간 기준)를 함께 건다

이것을 모르면 「커넥터는 RUNNING 인데 파일이 안 생긴다」로 한참 헤맨다

3. flush.size

너무 작으면 · 너무 크면 — 실습과 운영이 다른 이유

너무 작으면

작은 파일이 쏟아진다 — 5주차에서 본 그 문제

- 파일은 금방 생겨 보기엔 좋다
- 작은 파일이 쌓이면 나중에 읽기가 느리다
- 실습에서는 5 — 결과를 빨리 보려고

VS

너무 크면

파일이 늦게 생겨 지연이 커진다

- 운영에서는 수천~수만 건
- 시간 기준(rotate.interval.ms)을 병행
- 적재 후 주기적 컴팩션(rewrite_data_files)

4. 적재 경로

경로는 파티셔너가 정한다 — 기본과 날짜 기준

파티셔너	경로
기본 DefaultPartitioner	topics/<토픽명>/partition=<N>/ <토픽명>+<N>+<시작오프셋>.json
예	topics/shop.shop.orders/partition=0/ shop.shop.orders+0+0000000000.json
TimeBased Partitioner	topics/<토픽명>/year=2026/month=09/day=06/... 날짜가 경로에 들어간다

4-1 의 「슬래시는 폴더가 아니라 이름의 일부」— 날짜로 나눠 두면 Spark·Trino 가 필요한 날짜만 읽는다

5. raw 층

오늘 만드는 것은 raw 층 — 변환 없이 그대로

1. **Debezium 이벤트를 그대로**
   변환 없이 내려쓰는 raw 층 — 파일 안에 schema·payload 구조가 통째로 들어간다
2. **원본을 보존하는 이유**
   나중에 파싱할 수 있게 받은 그대로 둔다
3. **4-3 랜딩 존과 같은 발상**
   「랜딩 존은 받은 그대로」
4. **다음 단계는 Spark**
   이 파일들을 읽어 정제하고 Iceberg 테이블로 적재 — 8주차에서 이어진다

정리

이 1시간에서 꼭 가져갈 것

1. 싱크 커넥터는 코드 없는 컨슈머다
2. store.url 만 바꾸면 같은 커넥터가 AWS S3 와 MinIO 양쪽에 쓰인다
3. flush.size 만큼 모여야 파일이 생긴다 — 안 생긴다고 고장이 아니다
4. 작게 잡으면 작은 파일이 쏟아지고, 크게 잡으면 지연이 커진다
5. 적재 경로는 파티셔너가 정하고, 날짜로 나누면 읽기가 빨라진다
6. raw 층은 변환하지 않고 원본을 보존한다

6-5 · 이론 1H 마무리

다음은 S3 Sink 적재 실습 (2.5H)

MinIO 를 띄우고 싱크 커넥터를 등록해, DB 변경이 파일로 떨어지는 전 과정을 확인합니다

- ✓ 플러그인 마운트와 --force-recreate
- ✓ cdc-raw 버킷 만들기
- ✓ sink-minio.json 등록 → RUNNING
- ✓ INSERT 5건 → MinIO 파일 확인

준비할 것

6-4 스택 실행 중 · S3 Sink zip